

<https://killercoda.com/>
<https://hub.docker.com/>
<https://kubernetes.io/docs>

<https://training.linuxfoundation.org/certification/certified-kubernetes-administrator-cka/>

Login to the Linux VM and run the following commands -

1. `cat /etc/os-release` [`cat /etc/redhat-release` for RedHat specific OS]
2. `free -h` [Memory Statistics]
3. `lscpu` [CPU Information]
4. `lsblk` [Disk Information]
5. `uname -r` [kernel version]

Managing Containers using Docker

Software applications typically depend on other libraries, configuration files, or services provided by their runtime environment. Traditionally, the runtime environment for a software application is installed in an operating system running on a physical host or virtual machine. Any application dependencies are installed along with that operating system on the host.

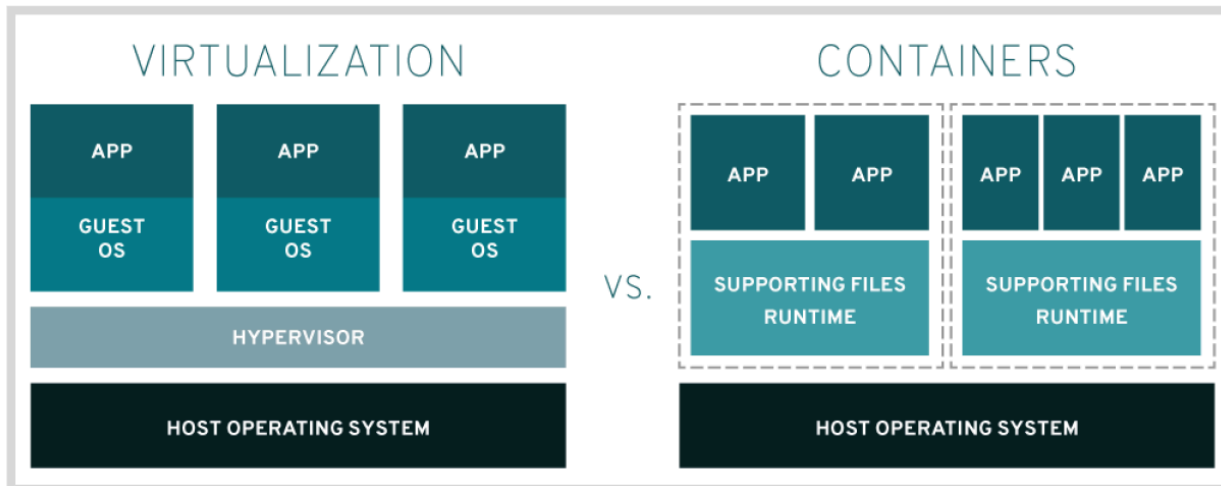
In Red Hat Enterprise Linux, packaging systems like RPM are used to help manage application dependencies. When you install the `httpd` package, the RPM system ensures that the correct libraries and other dependencies for that package are also installed.

The major drawback to traditionally deployed software applications is that these dependencies are entangled with the runtime environment. An application might require versions of supporting software that are older or newer than the software provided with the operating system. Similarly, two applications on the same system might require different versions of the same software that are incompatible with each other.

One way to resolve these conflicts is to package and deploy the application as a container. A container is a set of one or more processes that are isolated from the rest of the system.

Comparing Containers to Virtual Machines

Both technologies isolate their application libraries and runtime resources from the host operating system or hypervisor and vice versa.



Containers and Virtual Machines are different in the way they interact with hardware and the underlying operating system.

Virtualization:

- Enables multiple operating systems to run simultaneously on a single hardware platform.
- Uses a hypervisor to divide hardware into multiple virtual hardware systems, allowing multiple operating systems to run side by side.
- Requires a complete operating system environment to support the application.

Compare and contrast this with containers, which:

- Run directly on the operating system, sharing hardware and OS resources across all containers on the system. This enables applications to stay lightweight and run swiftly in parallel.
- Share the same operating system kernel, isolate the containerized application processes from the rest of the system, and use any software compatible with that kernel.
- Require far fewer hardware resources than virtual machines, which also makes them quick to start and stop and reduce storage requirements.

Running Containers from Container Images

- Containers are run from container images.
- Container images serve as blueprints for creating containers.
- Container images package an application together with all its dependencies, such as:
 - System libraries
 - Programming language runtimes
 - Programming language libraries
 - Configuration settings
 - Static data files

- Container images are unchangeable, or *immutable*, files that include all the required code and dependencies to run a container.

Container images are built according to specifications, such as the *Open Container Initiative (OCI)* image format specification. These specifications define the format for container images, as well as the metadata about the container host operating systems and hardware architectures that the image supports.

Designing Container-based Architectures

You could install a complex software application made up of multiple services in a single container. For example, you might have a web server that needs to use a database and a messaging system. But using one container for multiple services is hard to manage.

A better design runs each component, the web server, the database, and the messaging system, in separate containers. This way, updates and maintenance to individual application components do not affect other components or the application stack.

Container Images and Registries

- A container registry is a repository for storing and retrieving container images.
- Container images are uploaded, or pushed, to a container registry by a developer.
- We download, or pull, those container images from the registry to a local system so that you can use them to run containers.
- You might use a public registry containing third-party images, or you might use a private registry controlled by your organization.
- The source of your container images matters. Just like any other software package, you must know whether you can trust the code in the container image.
- Different registries have different policies about whether and how they provide, evaluate, and test container images submitted to them.
- Red Hat distributes certified container images through two main container registries that you can access with your Red Hat login credentials.

registry.redhat.io for containers based on official Red Hat products.

registry.connect.redhat.com for containers based on third-party products.

- Red Hat is gradually phasing out an older registry, **registry.access.redhat.com**.

To delete all images ⇒ `docker rmi $(docker images -q -a)`

Container Image Naming Conventions

Container images are named based on the following fully qualified image name syntax:

`registry_name/user_name/image_name:tag`

`registry.redhat.io/rhel8/httpd-24:latest`

- The registry_name is the name of the registry storing the image. It is usually the fully qualified domain name of the registry.
- The user_name represents the user or organization to which the image belongs.
- The image_name must be unique in the user namespace.
- The tag identifies the image version. If the image name includes no image tag, then the latest is assumed.

Registry Configuration File : /etc/containers/registries.conf

Install Docker package

For Amazon Linux / CentOS Stream 9

```
sudo yum update
sudo yum install docker
sudo systemctl enable --now docker
sudo systemctl status docker
sudo docker version
```

For Ubuntu

Reference : <https://docs.docker.com/engine/install/ubuntu/>

RedHat Container Registry Authentication :

<https://access.redhat.com/articles/RegistryAuthentication>

Running Containers

- To run a container on your local system, you must first pull a container image.
- The docker pull command pulls the image you specify from the registry and saves it locally:

```
docker pull ubuntu OR docker pull docker.io/ubuntu
podman pull ubuntu
```

- Podman stores images locally and you can list them using the podman images command:

```
docker images
podman images
```

To list the running containers -

```
docker ps
podman ps
```

To list the running and exited containers -

```
docker ps -a
podman ps -a
```

- To run a container from this image, use the docker run command.
docker run -it --name=myubuntu <image_name> /bin/bash

```
cat /etc/os-release
apt update
apt install python3
apt install git
python3 --version
git --version
exit
```

To execute a command inside a container ⇒ `docker exec -t <container_name>`

`uname -r`

To stop a container ⇒ `docker stop <container_name>`

To start a container ⇒ `docker start <container_name>`

To remove a container ⇒ `docker rm <container_name>`

To remove a container without stopping it ⇒ `docker rm <container_name> --force`

To remove a container image ⇒ `docker rmi <image_name>`

Run an Apache httpd web container

```
docker pull httpd
```

```
docker images
```

```
docker run --name web -d -it httpd [ -d ⇒ detached mode ]
```

```
docker inspect web [ note the IP address and Gateway of the container ]
```

```
ifconfig [ note the docker0 virtual interface which acts as the default gateway for all containers ]
```

Also note the **veth** interface created for each container.

Access the container ⇒ `curl http://container\_IP` ⇒ Displays the Home Page

How can we access the container from outside the container host?

Using Port Mapping [*Host Port : Container Port*]

```
docker run --name web -d -p 80:80 httpd
```

Container Storage - is ephemeral i.e. non-persistent.

Provide persistent storage to the container ⇒

```
mkdir -p webcontent/html
```

```
echo "Hello nginx" > webcontent/html/index.html
```

```
docker run --name web -d -p 80:80 -v ~/webcontent:/usr/share/nginx nginx
```

Building a Kubernetes Cluster

Relevant Documentation

Installing kubeadm ⇒

<https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/>

Creating a cluster with kubeadm ⇒

<https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/create-cluster-kubeadm/>

Lesson Reference

Distribution: Ubuntu 24.04 LTS

VM Size: medium

To check the releases ⇒ <https://kubernetes.io/releases/>

1. You will need to load some kernel modules and modify some system settings as part of this process on all nodes.

```
cat << EOF | sudo tee /etc/modules-load.d/containerd.conf
overlay
br_netfilter
EOF
```

```
modprobe overlay
modprobe br_netfilter
```

```
cat <<EOF | sudo tee /etc/sysctl.d/99-kubernetes-cri.conf
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
EOF
sysctl --system
```

2. On all nodes, Install and configure containerd.

```
apt-get update && apt-get install -y containerd
mkdir -p /etc/containerd
containerd config default | tee /etc/containerd/config.toml
apt-get install gpg vim -y
```

vim /etc/containerd/config.toml

```
[plugins.'io.containerd.cri.v1.runtime'.containerd.runtimes.runc.options]
  BinaryName = ''
  CriuImagePath = ''
  CriuWorkPath = ''
  IoGid = 0
  IoUid = 0
  NoNewKeyring = false
  Root = ''
  ShimCgroup = ''
  SystemdCgroup = true
```

SystemdGroup = false should be changed to true
systemctl restart containerd
systemctl status containerd

3. On all nodes, disable swap.

swapoff -a

4. On all nodes, install kubeadm, kubelet, and kubectl.

```
apt-get update && sudo apt-get install -y apt-transport-https curl
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.32/deb/ /" | sudo tee
/etc/apt/sources.list.d/kubernetes.list
```

```
mkdir -p /etc/apt/keyrings
```

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.32/deb/Release.key | sudo gpg --dearmor
-o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

```
apt-get update
apt-cache madison kubeadm
```

```
apt-get install -y kubelet=1.32.1-1.1 kubeadm=1.32.1-1.1 kubectl=1.32.1-1.1
```

```
apt-mark hold kubelet kubeadm kubectl
```

5. On the control plane node only, initialize the cluster and set up kubectl access.

```
kubeadm init --pod-network-cidr 192.168.0.0/16 --kubernetes-version 1.32.1
mkdir -p $HOME/.kube
cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
chown $(id -u):$(id -g) $HOME/.kube/config
```

Verify the cluster is working.

```
systemctl status kubelet
kubectl get nodes
kubectl version
kubeadm version
kubelet --version
```

6. On the control plane node only install the Calico network add-on

[Reference : <https://docs.tigera.io/calico/3.25/getting-started/kubernetes/quickstart>]

```
kubectl create -f
https://raw.githubusercontent.com/projectcalico/calico/v3.25.0/manifests/tigera-operator.y  
aml
```

```
kubectl create -f
https://raw.githubusercontent.com/projectcalico/calico/v3.25.0/manifests/custom-resources  
.yaml
```

```
kubectl get nodes
```

7. Get the join command (this command is also printed during kubeadm init . Feel free to simply copy it from there).

```
kubeadm token create --print-join-command
```

8. Copy the join command from the control plane node. Run it on each worker node as root (i.e. with sudo).

```
sudo kubeadm join ...
```

9. On the control plane node, verify all nodes in your cluster are ready. Note that it may take a few moments for all of the nodes to enter the READY state.

```
kubectl get nodes
```

Kubernetes Official Doc : <https://kubernetes.io/docs>

Kubernetes Components : <https://kubernetes.io/docs/concepts/overview/components/>

CRI,Kubelet,KubectI,Kubeadm :

<https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/>

Kubernetes Architecture Diagram : <https://kubernetes.io/docs/concepts/architecture/>

Get information about the Kubernetes Cluster

1. Check the cluster nodes ⇒ `kubectl get nodes`
2. Check the version of `kubectl` and `kubeadm` ⇒ `kubectl version` ⇒ `kubeadm version`
3. List all resources in the cluster ⇒ `kubectl get all`
4. To list the pods ⇒ `kubectl get pods`
5. Kubernetes Resources ⇒ Pod, ReplicationController, ReplicaSet, DaemonSet, Deployment, PersistentVolume[PV], PersistentVolumeClaim[PVC], Service
6. To display info about a resource ⇒ `kubectl describe pod/pod_name`
7. Access a Pod ⇒ `kubectl exec -it pod_name – bash`

Kubernetes is an open source container orchestration engine for automating deployment, scaling, and management of containerized applications. The open source project is hosted by the Cloud Native Computing Foundation (CNCF).

The name Kubernetes originates from Greek, meaning helmsman or pilot. K8s as an abbreviation results from counting the eight letters between the "K" and the "s". Google open sourced the Kubernetes project in 2014.

Container Orchestration is the automated process of managing and coordinating containers, which are lightweight, portable units that package applications and their dependencies. This process involves deploying, scaling, and managing containerized applications across multiple environments.

Key aspects of container orchestration include:

1. **Deployment:** Orchestrators automate the deployment of containers across a cluster of machines, ensuring that they are launched in the correct sequence and in the appropriate environment.
2. **Scaling:** As demand fluctuates, container orchestration platforms can automatically scale the number of running containers up or down to meet performance requirements.

3. **Load Balancing:** Orchestration tools distribute network traffic across containers to ensure that no single container is overwhelmed with requests, improving application performance and availability.
4. **Health Monitoring:** Orchestrators continuously monitor the health of containers, automatically restarting or replacing failed containers to maintain the desired state of the application.
5. **Networking:** They manage communication between containers and other services, often creating virtual networks that allow containers to communicate securely with each other.
6. **Storage Management:** Orchestrators handle the storage needs of containers, ensuring that data is persisted and accessible even if a container is moved or restarted.

Popular container orchestration tools include ***Kubernetes, Docker Swarm, and Apache Mesos***. Kubernetes is the most widely used and has become the standard for container orchestration due to its robust features and scalability.